

Advanced Data Management – Week 1

Introduction to Data Management and Relational Systems

Lecture Objectives

- By the end of this lecture, students should be able to:
- Explain the historical evolution of data management systems
- Analyze the limitations of Relational Database Management Systems (RDBMS) in large-scale environments
- Describe the fundamental characteristics of Big Data (Volume, Velocity, Variety)
- Distinguish the architectural and philosophical differences between relational and NoSQL databases
- Justify why database selection is an architectural decision in modern distributed systems

Course Context & Evolution of Data Management

Transition from file-based systems to DBMS

Dominance of relational databases

Emergence of large-scale distributed applications

Rise of Big Data and NoSQL architectures

What is Data Management?

- Data Management refers to the set of principles, processes, technologies, and practices that ensure that an organization's data are **collected, stored, organized, protected, and utilized** in a reliable, efficient, and secure manner.
- Its objective is not simply the storage of data, but their transformation into useful, reliable, and **reusable** information throughout their lifecycle.

What “Data Management” Means

The term data management includes a broad range of activities such as:

- Designing the structure of data
- Storing and retrieving data
- Ensuring quality, consistency, and integrity
- Security and access control
- Compliance with regulatory frameworks
- Supporting analytics and decision-making

In other words, data management answers the question: How do we manage data so that they are always available, correct, and useful when required by the information system?

Why Data Are a Central Element of Every Information System

Every information system, regardless of application or domain, has as its primary purpose the management of data.

- Applications:
 - Input data
 - Process them
 - Produce output
- Without data:
 - There is no information
 - No automation
 - No knowledge
 - No decision support

Data constitute the permanent and long-term component of a system, unlike applications that change or are replaced. For this reason, their proper management is critical for the sustainability, reliability, and scalability of every information system.

Role of the Database as an Intermediate Layer Between Application and Storage

The database functions as the intermediate layer (middleware) between:

- Applications (logic and user interface)
- Physical storage media (disks, cloud storage, distributed systems)

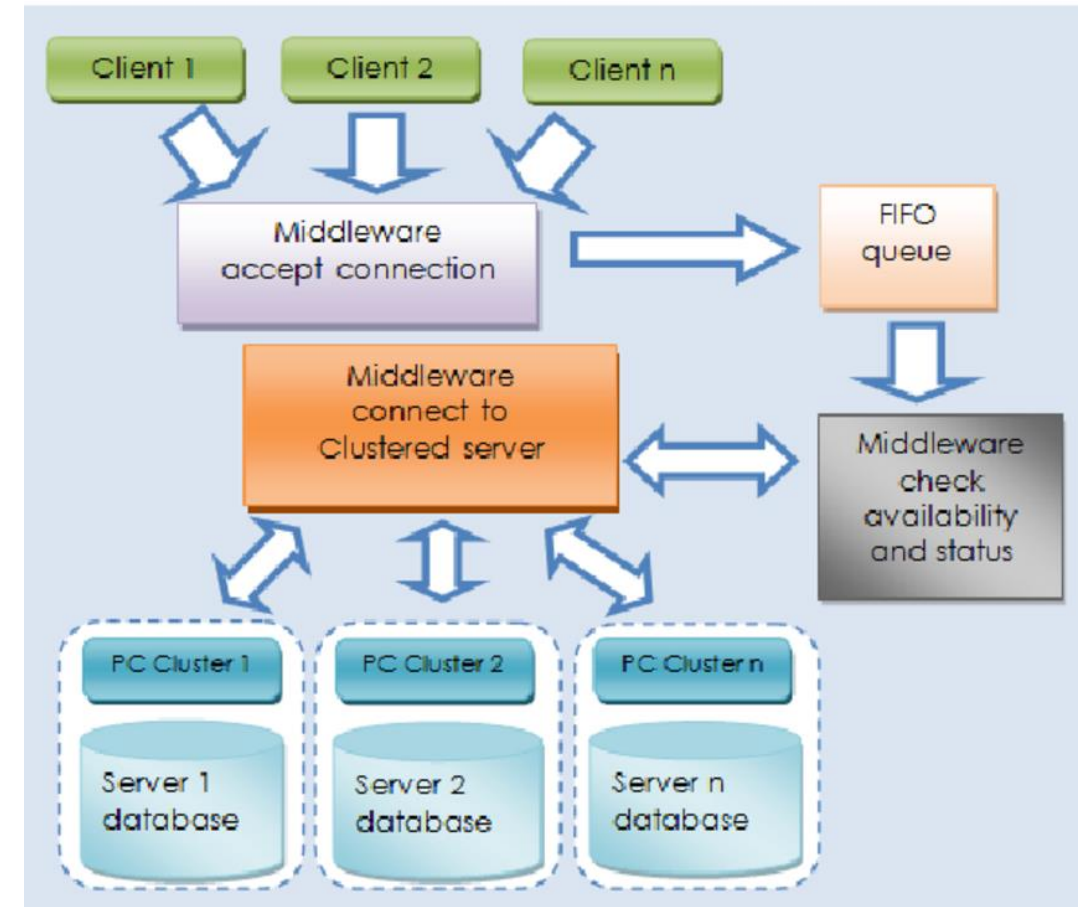
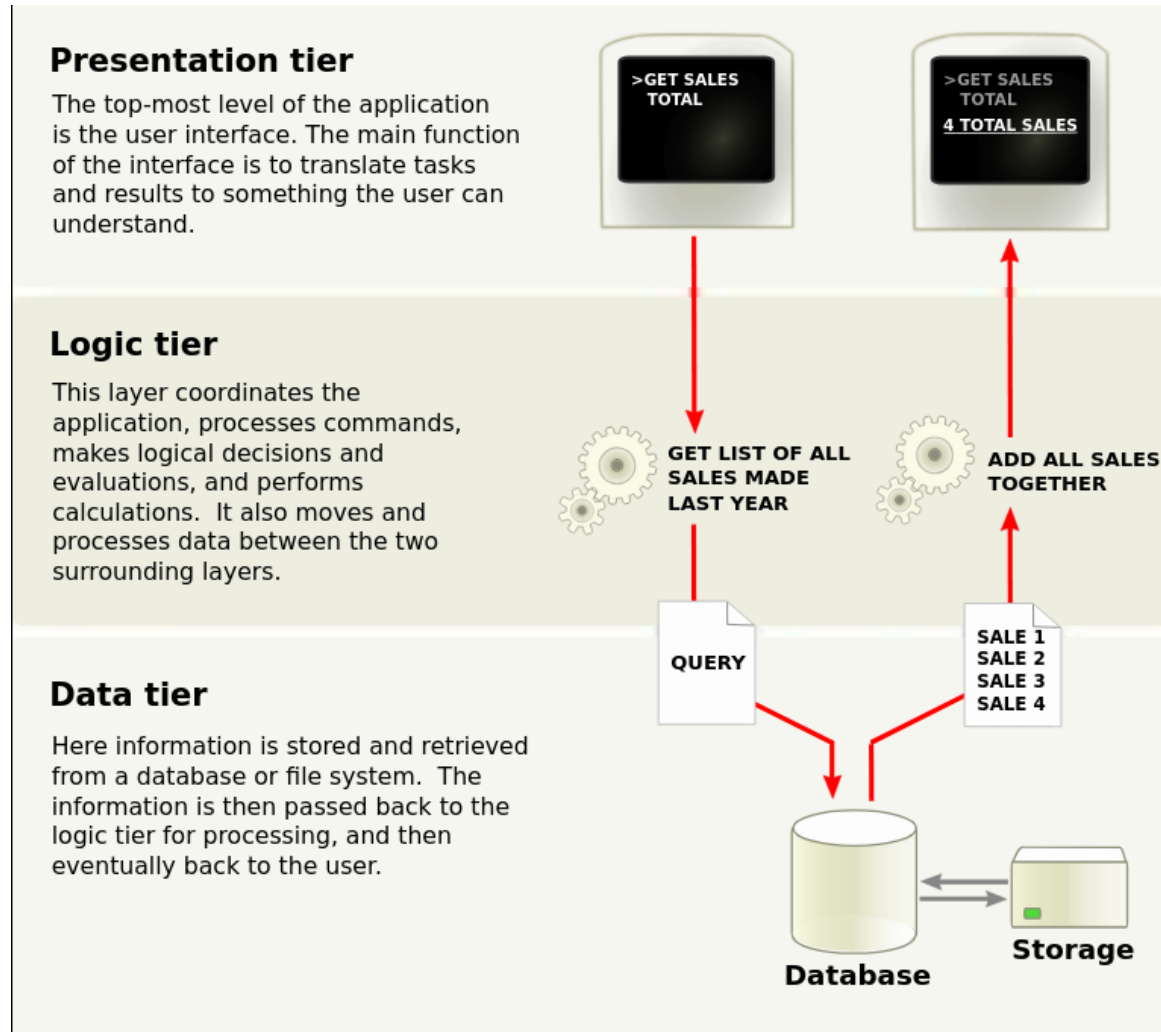
Its role is crucial because it:

- Hides physical storage details from the application
- Provides a unified and structured way to access data
- Guarantees integrity, consistency, and concurrent multi-user access
- Implements security, transaction, and recovery mechanisms

Thus, the application does not directly manage files or disks, but communicates with the database, which handles all the complexity of storage and data management.

In conclusion, data management constitutes the foundation of every modern information system, with the database serving as the reliable and controlled connector between application logic and underlying storage infrastructure.

Role of the Database as an Intermediate Layer Between Application and Storage



File-based Systems

- Before the emergence of Database Management Systems (DBMS), most information systems relied on file-based systems.
- In this approach, each application managed its data directly through operating system files, without an intermediate management layer.
- File-based systems represented the first step in organizing digital data, but the increasing complexity of applications quickly exposed their structural limitations.

Storage of Data in Files

In file-based systems:

- Data are stored in simple files (e.g., text, binary, CSV)
- Each application defines its own file structure
- Reading and writing are performed directly by application code
- There is no central control or integration mechanism

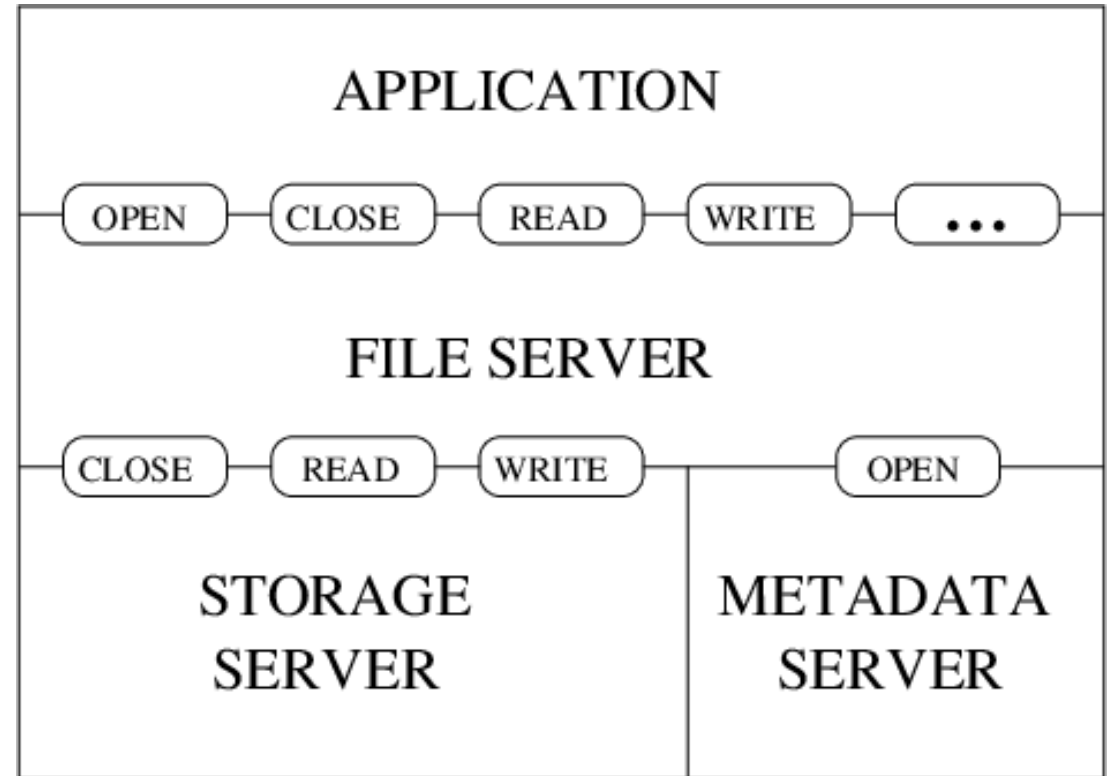
The file simultaneously serves as both storage medium and logical data model.

Tight Coupling Between Data and Code

A central characteristic of file-based systems is tight coupling:

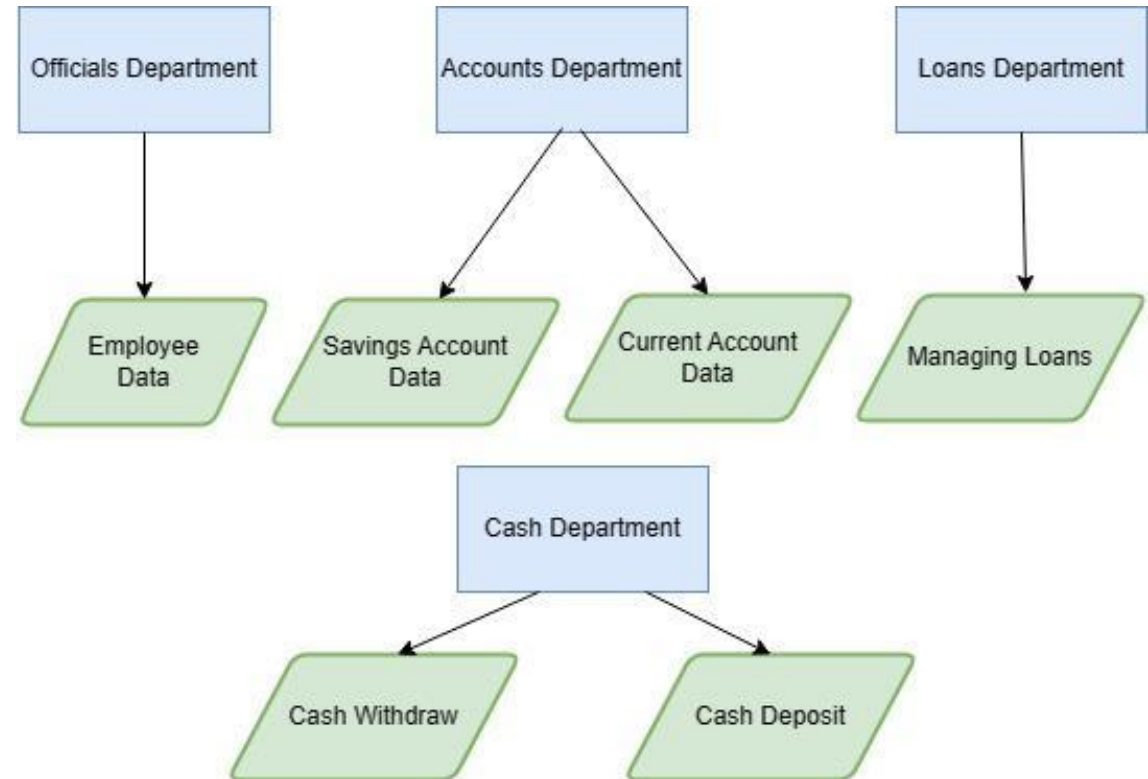
- The data structure is hard-coded into the application
- Any change in file format requires code modifications
- The application “knows” storage details

This means that data are not independent from the application that uses them.



Problems of File-based Systems

- Redundancy (Πλεονασμός δεδομένων)
- Inconsistency (Ασυνέπεια δεδομένων)
- Maintenance difficulty
- Lack of concurrent access



File-based Systems – Redundancy

The same data are often stored in multiple files for different applications.

This leads to:

- Waste of storage space
- Increased probability of errors
- Difficulty in synchronizing information

File-based Systems – Inconsistency

When the same data exist in multiple files:

- Updates may be applied only to some of them
- Conflicting values arise
- System reliability decreases

File-based Systems – Maintenance Difficulty

Tight coupling makes maintenance particularly difficult:

- Small changes in data structure require extensive code changes
- Development and evolution costs increase
- System scalability decreases

File-based Systems – Lack of Concurrency

File-based systems do not effectively support multiple users:

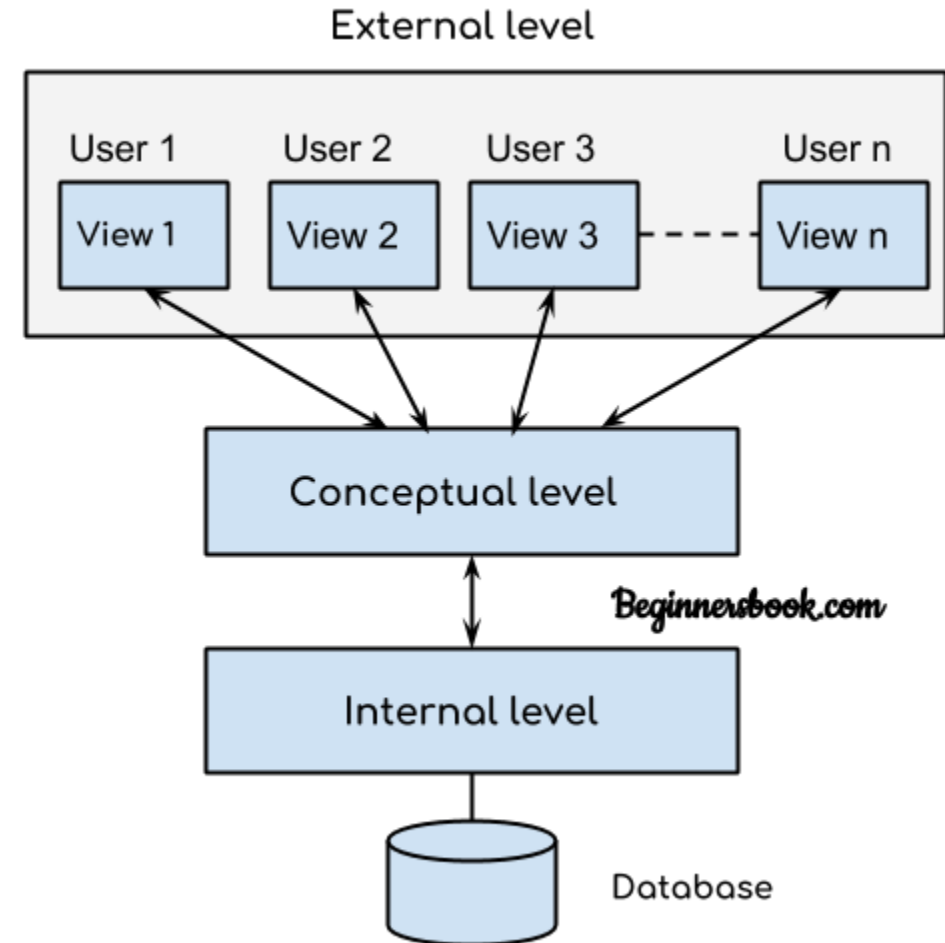
- No concurrency control mechanisms
- Conflicts during simultaneous writes
- Risk of data loss or corruption

File-based systems were suitable for simple, single-user, small-scale applications.

- However, redundancy, inconsistency, maintenance, and concurrency problems made it clear that a more structured and independent approach was necessary.
- This need led directly to the development of Database Management Systems (DBMS), which address precisely these weaknesses.

Emergence of DBMS (Database Management Systems)

- The emergence of Database Management Systems (DBMS) was a direct response to the structural problems of file-based systems.
- DBMS introduced an intermediate software layer between applications and storage, centrally managing data.
- Thus, data cease to be the property of a single application and become a shared, controlled, and independent resource.



DBMS – Abstraction

The concept of abstraction is fundamental in DBMS.

The system:

- Hides the details of how data are stored
- Provides users and applications with a simplified logical view of data
- Separates physical storage, logical structure, and access levels

Through abstraction, applications interact with data via commands and interfaces, without knowing where or how they are physically stored.

DBMS – Data Independence

Data independence means that:

- Changes in storage mechanisms (files, indexes, partitions) do not require changes in application code
- Changes in logical structure can occur without affecting users

It is distinguished into:

- **Physical Data Independence:** Independence from physical storage details.
- **Logical Data Independence:** Independence from schema changes.

This represents a fundamental advantage over file-based systems.

DBMS – Centralized Data Management

With DBMS:

- Data are stored and controlled centrally
- Data are shared among multiple applications and users
- Unified integrity, security, and access rules are enforced

Centralized management reduces redundancy, increases consistency, and enables holistic supervision of the information system.

DBMS – Core Functions 1/3

CRUD (Create, Read, Update, Delete)

DBMS provide standardized operations for:

- Data insertion
- Information retrieval
- Record updating
- Data deletion

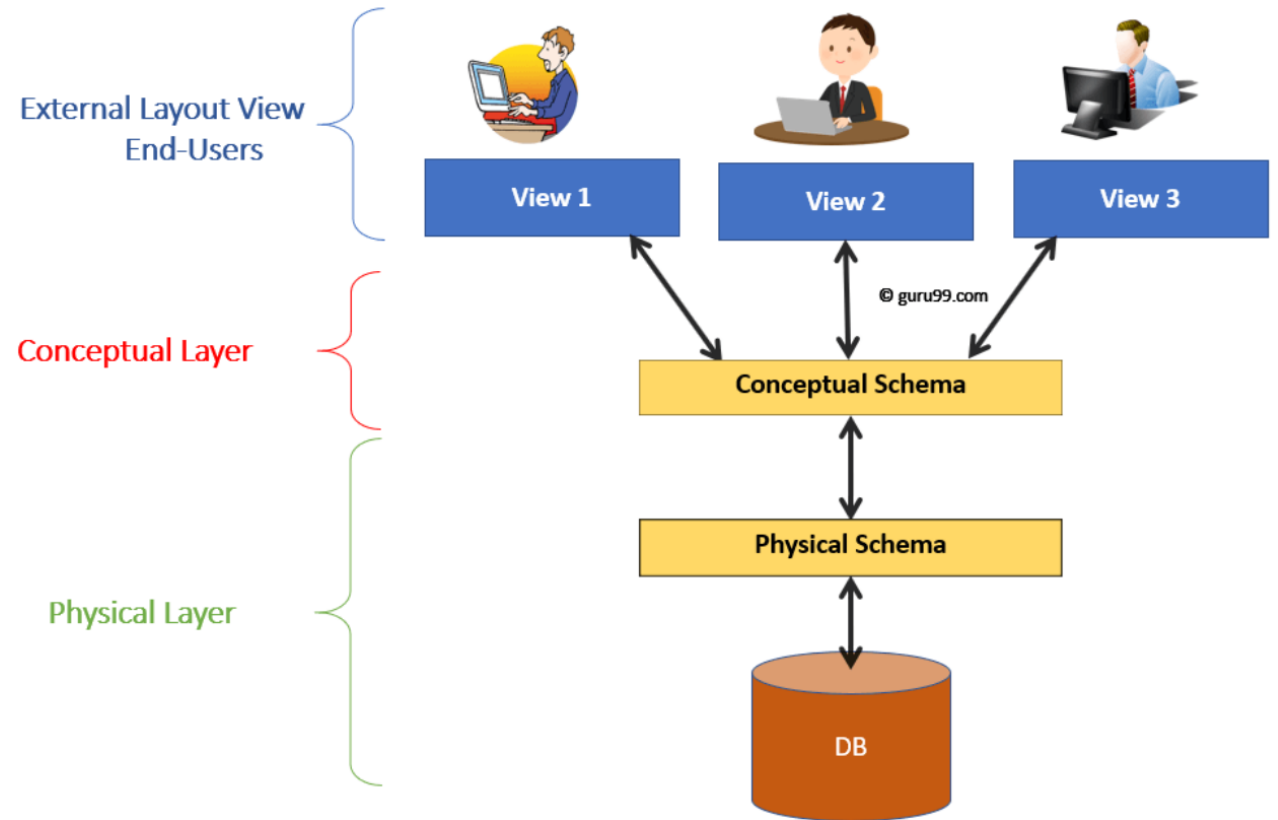
These operations constitute the core of database interaction.

DBMS – Core Functions 2/3

Concurrency Control

DBMS allow safe concurrent access by multiple users:

- Prevent write conflicts
- Ensure data consistency
- Implement locking and transaction mechanisms



DBMS – Core Functions 3/3

Recovery

In case of failure:

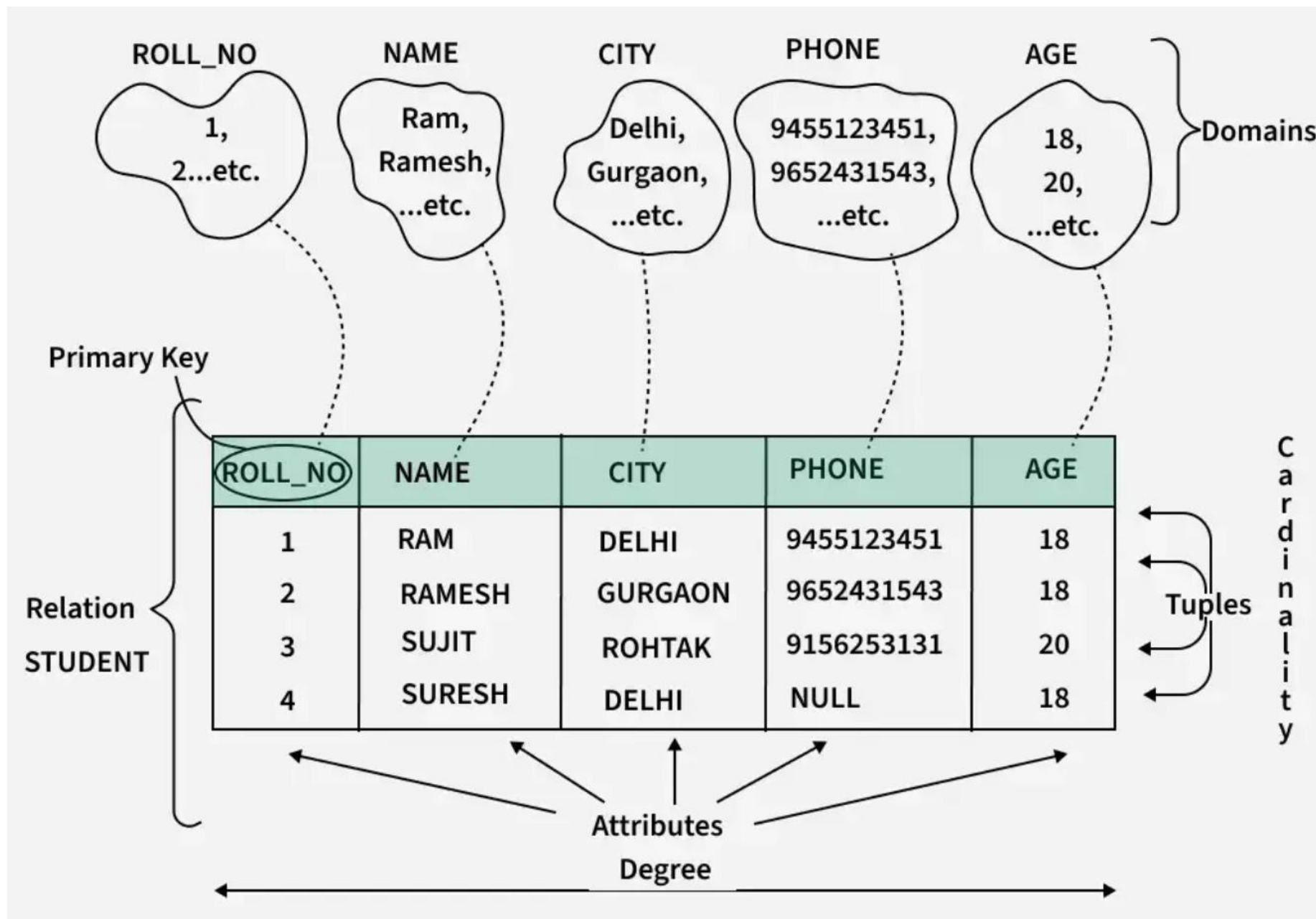
- The DBMS can restore the database to a consistent state
- Uses logs and rollback mechanisms
- Protects data from loss or partial writes

Conclusion (DBMS Section)

- The emergence of DBMS marked the transition from applications that “owned” data to systems where data are an autonomous, centrally managed resource.
- Abstraction, data independence, and core functions such as CRUD, concurrency control, and recovery constitute the cornerstone of every modern information system.

The Dominance of Relational Databases

- Relational databases prevailed as the dominant model of data storage and management for decades because they provided a strict, mathematically grounded, and universally understandable **way of organizing data**.
- The relational model offered clear structure, formal rules, and a high degree of reliability — essential characteristics for enterprise and mission-critical systems.



Relational Model

- The relational model is based on set theory and relational algebra.
- Data are represented as sets of tuples organized into relations.
- Each relation:
 - Has a clearly defined schema
 - Consists of attributes with specified data types
 - Is subject to integrity constraints
- This strict definition enabled data independence and rigorous consistency enforcement.

Tables, Relations, Keys

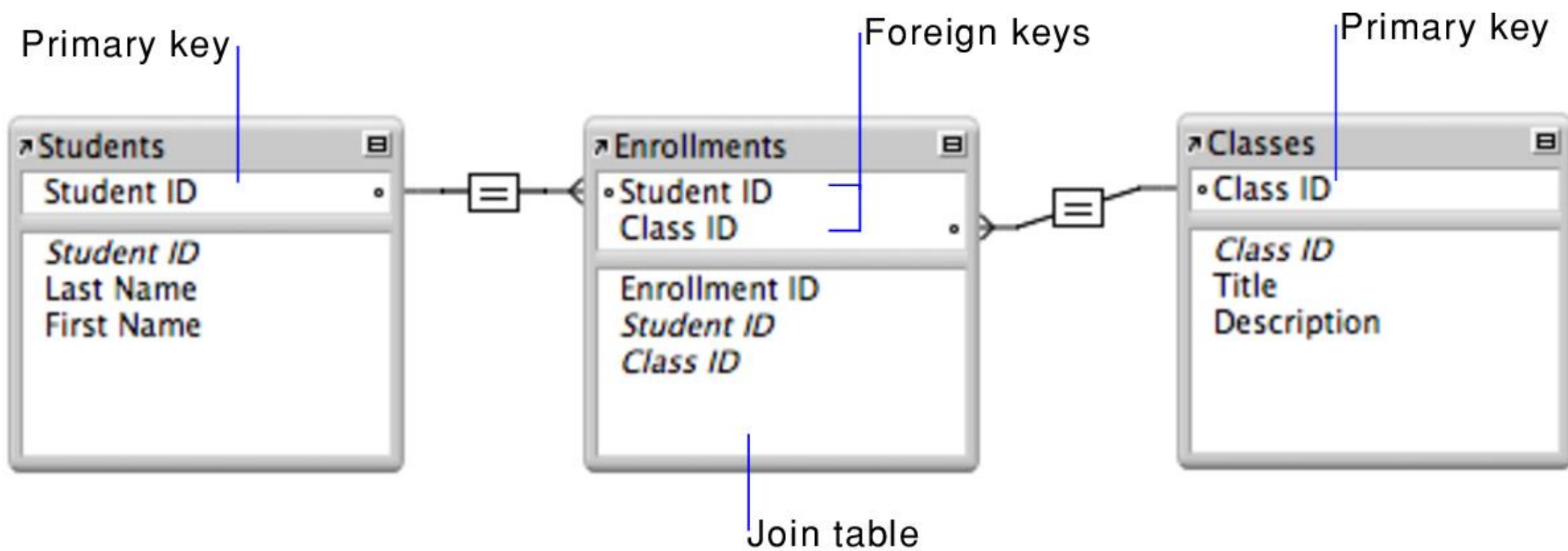
In practical implementation:

- Tables correspond to relations
- Rows represent records (entities)
- Columns represent attributes

Keys ensure uniqueness and relationships:

- **Primary Key:** Unique identifier of each record
- **Foreign Key:** Link between tables
- **Constraints:** Rules that preserve data integrity

In this way, complex data structures are modeled in a simple and standardized manner.



SQL as a Universal Language

SQL (Structured Query Language) was established as the universal language for interacting with relational databases.

It provides:

- Declarative query specification
 - Independence from physical storage
 - Unified standard for CRUD operations
 - Schema definition and access control
-
- The user describes what needs to be retrieved, not how, leaving execution optimization to the DBMS.

ACID Properties

The reliability of relational databases is based on the ACID properties of transactions:

- **Atomicity:** Each transaction is executed entirely or not at all.
- **Consistency:** The database transitions from one valid state to another.
- **Isolation:** Concurrent transactions do not interfere with each other.
- **Durability:** Data remain stored even after system failure.

ACID properties make relational databases suitable for banking, accounting, and other critical systems.

Relational Databases – Conclusion

The dominance of relational databases was not accidental.

The combination of:

- A strict relational model
- The universal SQL language
- ACID guarantees
- created a stable, reliable, and scalable foundation for large-scale information systems.
- This background explains why relational databases remain a reference point even in the modern era of NoSQL systems.

Limits of RDBMS & Big Data

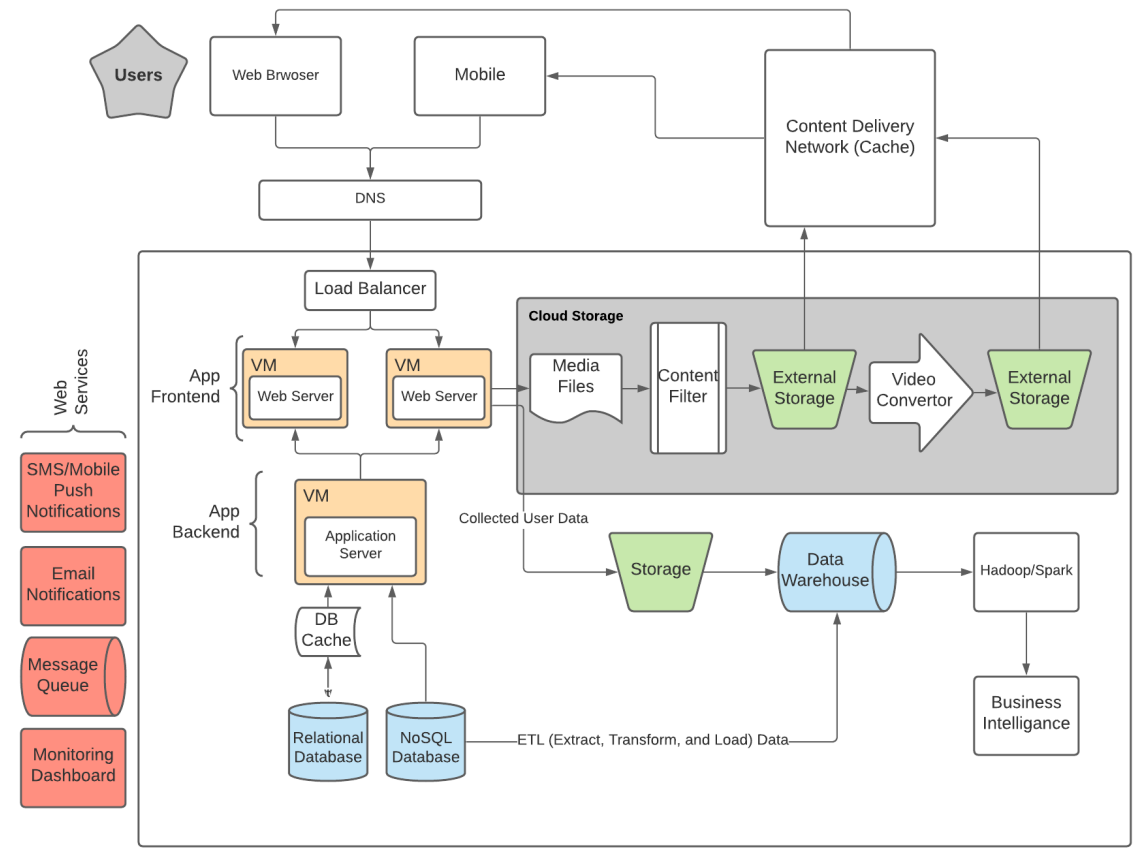
What Changed in the Application World

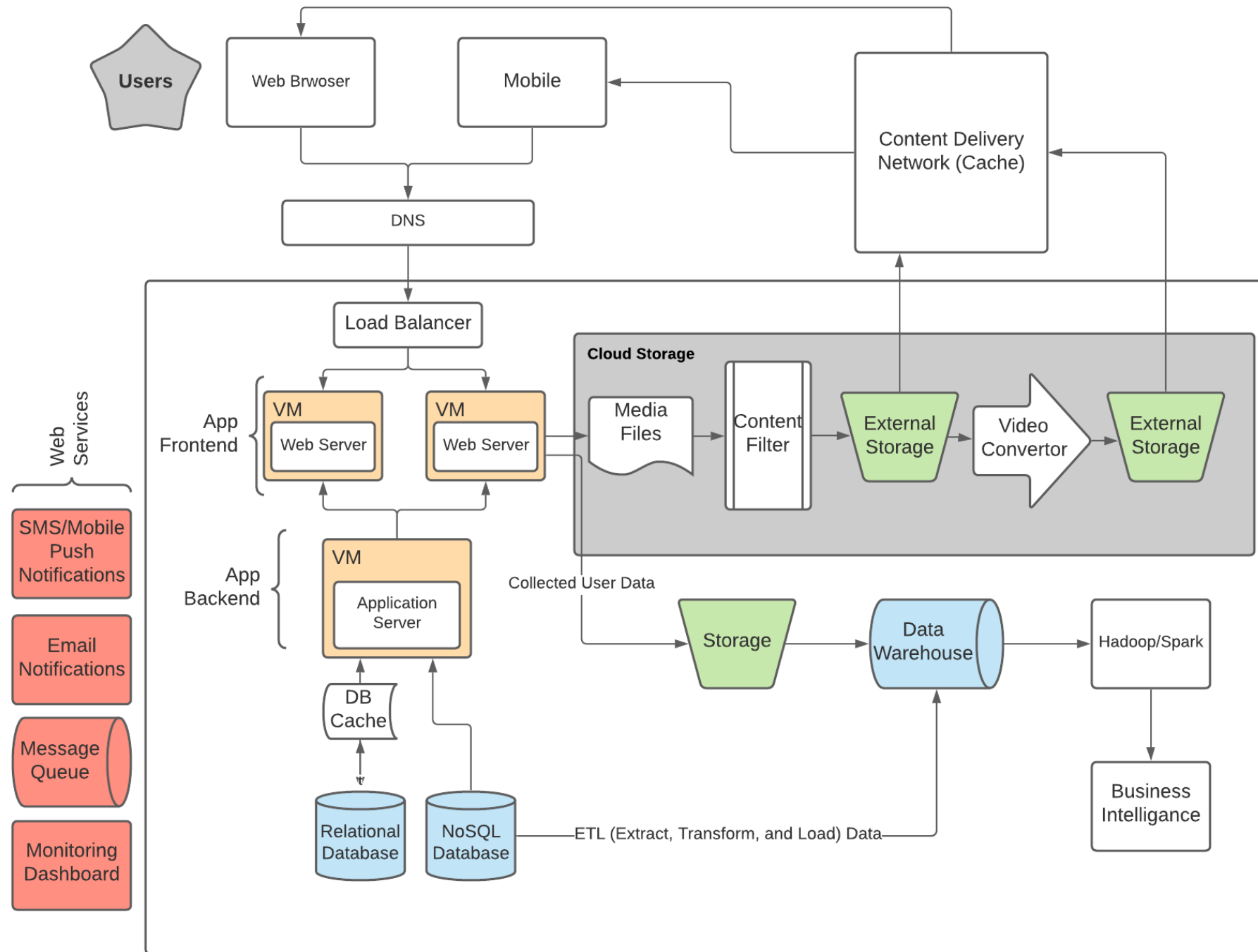
- The classical era of information systems was based on **relatively stable, small-scale, and internal applications**.
- Today, the application environment has been radically transformed, leading to **explosive growth in data volume, number of users, update rate, and system complexity** — often beyond the design assumptions of traditional RDBMS.

Web Applications

Applications moved from local systems to **globally accessible web platforms, serving millions of concurrent users.**

- They are characterized by:
 - High concurrency
 - Intensive read/write traffic
 - Need for horizontal scalability
 - Low latency requirements



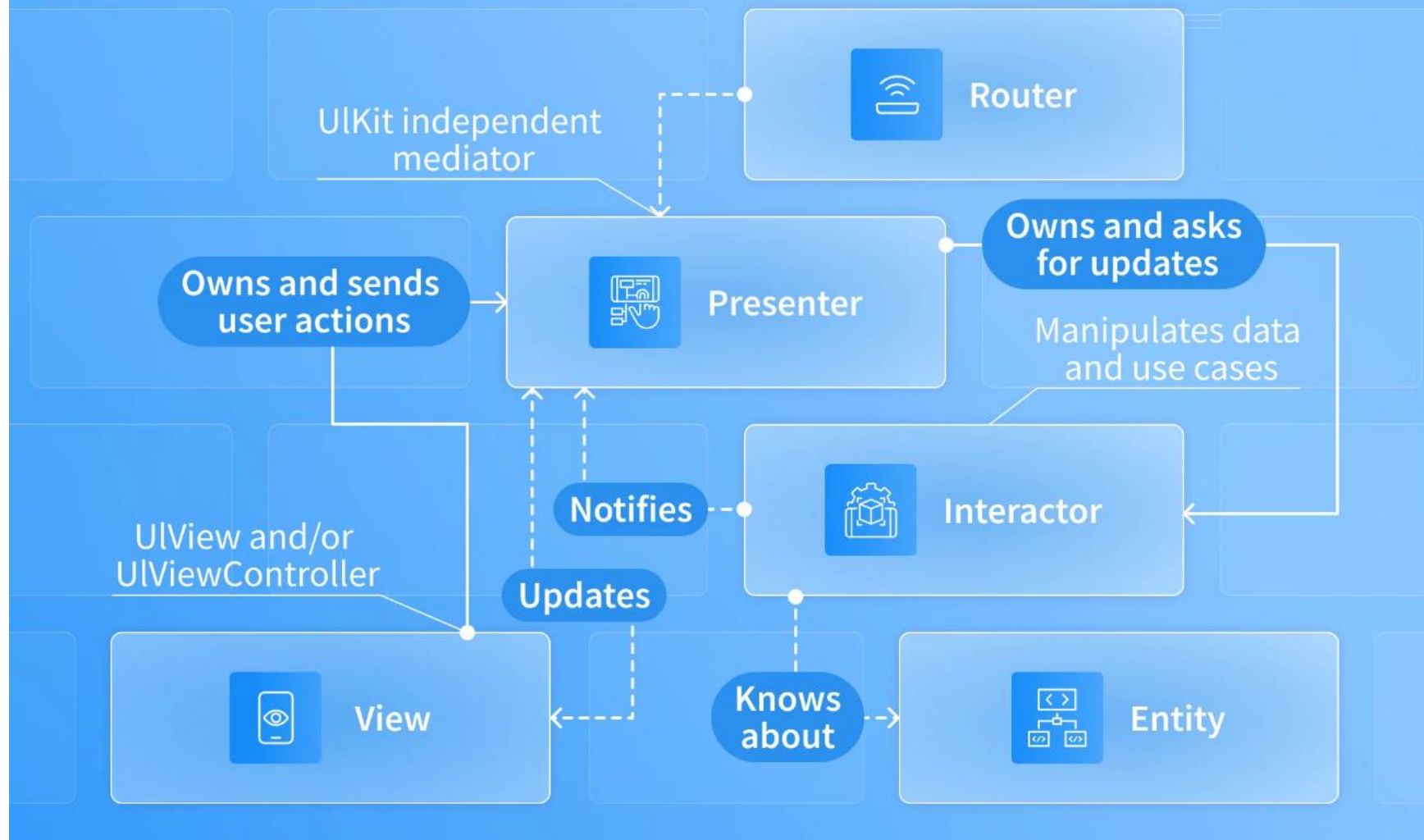


Mobile Users

The dominance of smartphones introduced:

- Continuous real-time data generation
- Geospatial information
- Asynchronous and unstable connectivity
- Demand for immediate response
- Offline-first architectures
- **Databases are now required to serve mobile, distributed, and dynamic data flows.**

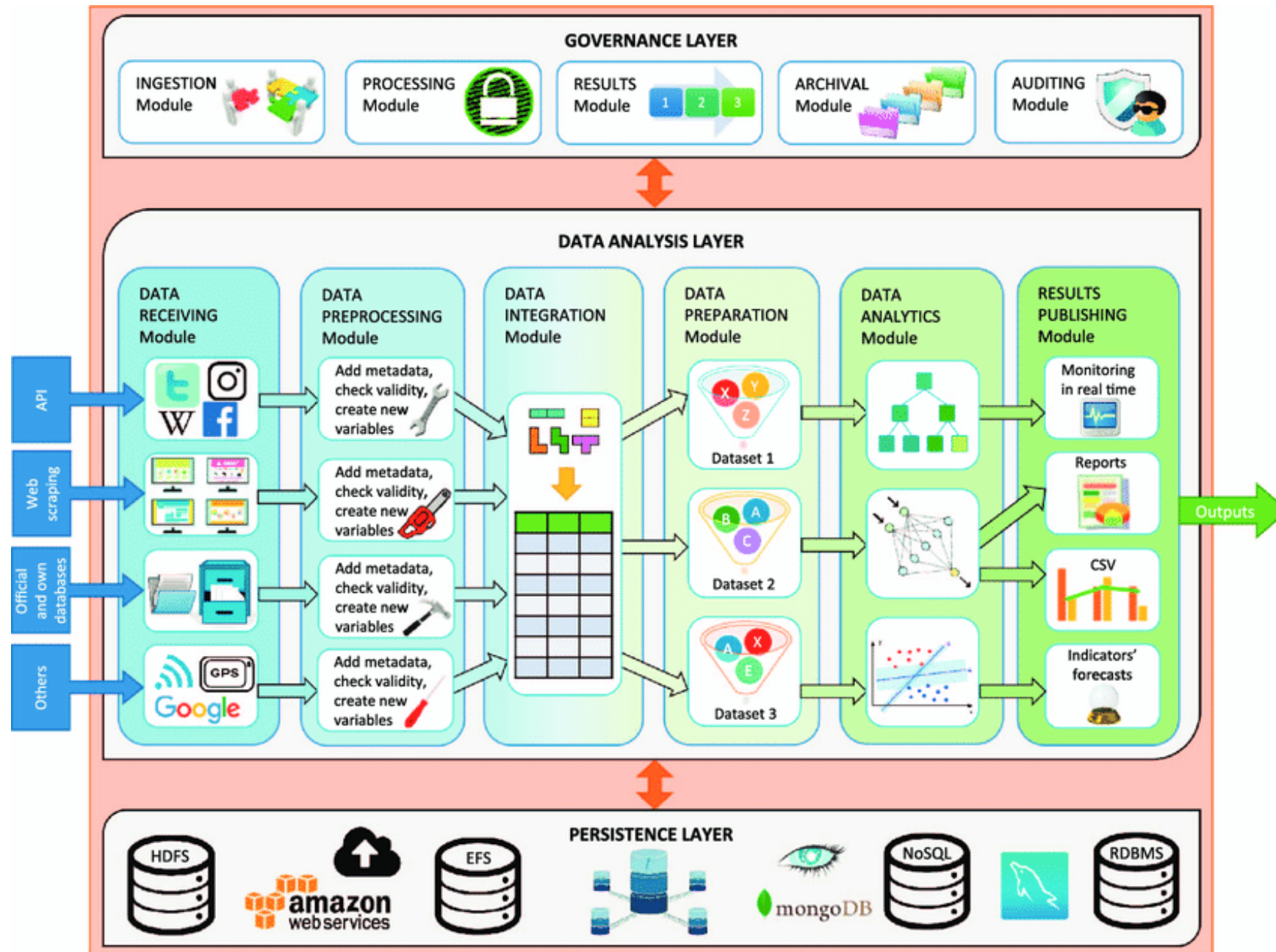
View, Interactor, Presenter, Entity, and Router Model (VIPER) Pattern



Social Networks

Social networks generated:

- **Massive volumes of unstructured data (posts, media, reactions)**
- Graph relationships between users
- Complex recommendation and graph queries
- Extremely high write throughput
- **Data are no longer only tables — they are graph structures, streams, and dynamic social graphs.**



Cloud Computing

The transition to cloud computing imposed:

- **Distributed data storage across multiple data centers**
- **Elastic scaling**
- Node failure tolerance
- Geographic distribution of users and workloads
- **Databases must operate in distributed, fault-tolerant, multi-region environments.**

IoT and Sensors

The expansion of the Internet of Things introduced:

- **Continuous data streams from millions of devices**
- High-frequency time-series data
- Need for real-time ingestion and analytics
- Huge volume of low-value-per-record data but extremely high overall scale
- **Data are now generated automatically, continuously, and in real time.**

Conclusion

- The world of applications **shifted from static, small-scale, and structured data usage to a massive, distributed, high-speed, and heterogeneous environment.**
- This transformation is the primary **reason traditional RDBMS reach their limits** and it paved the way for **Big Data platforms and NoSQL architectures.**

New Requirements

- The radical transformation of the application ecosystem created a set of new, non-negotiable requirements for data systems.
- These requirements no longer concern only correct storage, but continuous, global, and immediate access under extreme scale.

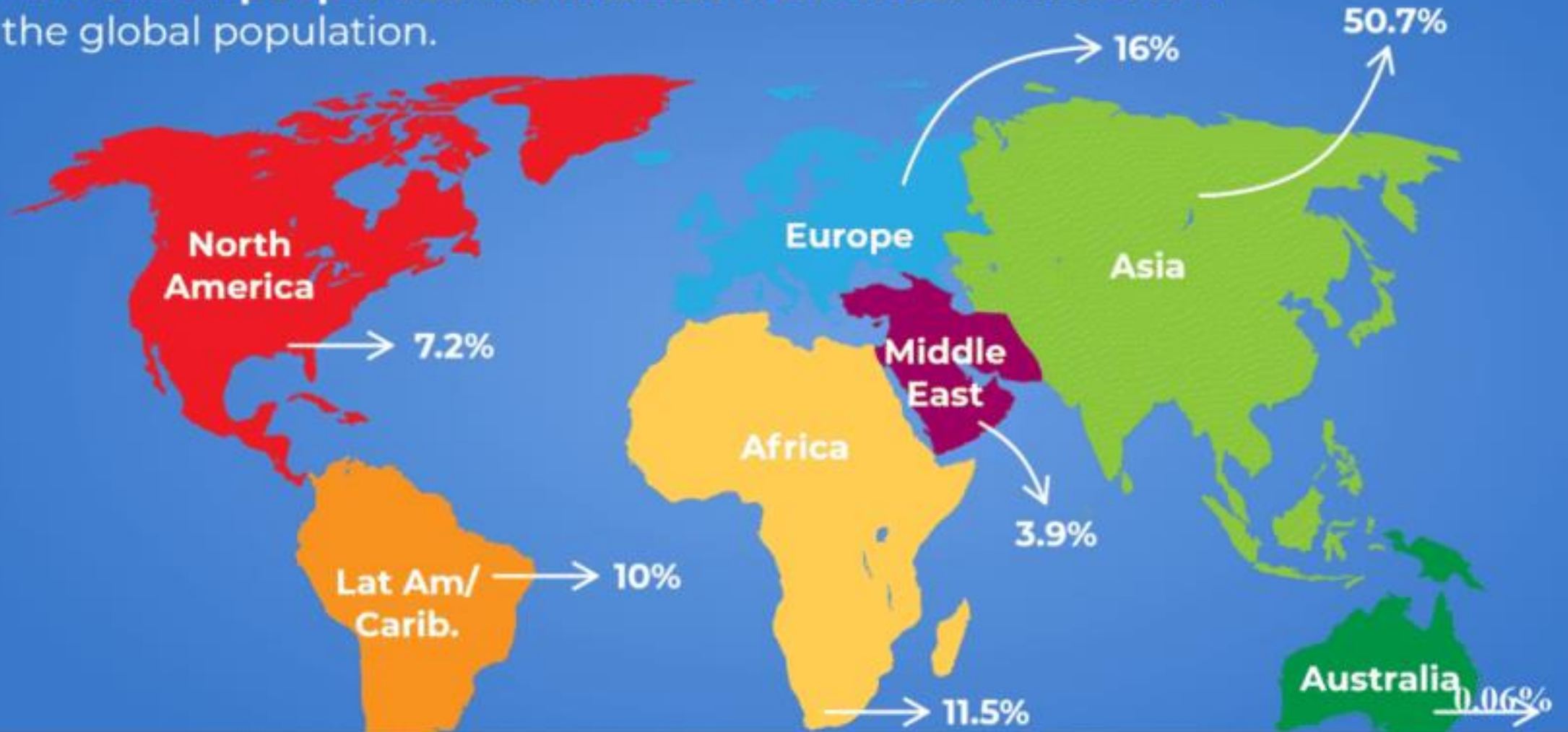
New Requirements – Massive Number of Users

Modern systems serve:

- Millions or even billions of users
- High levels of concurrency
- Highly variable workloads
- The classical RDBMS architecture proves insufficient, as scalability must be **horizontal and dynamic**.

Total Internet Users Worldwide Statistic

4.79 billion people use the internet worldwide. That's **57%** of the global population.



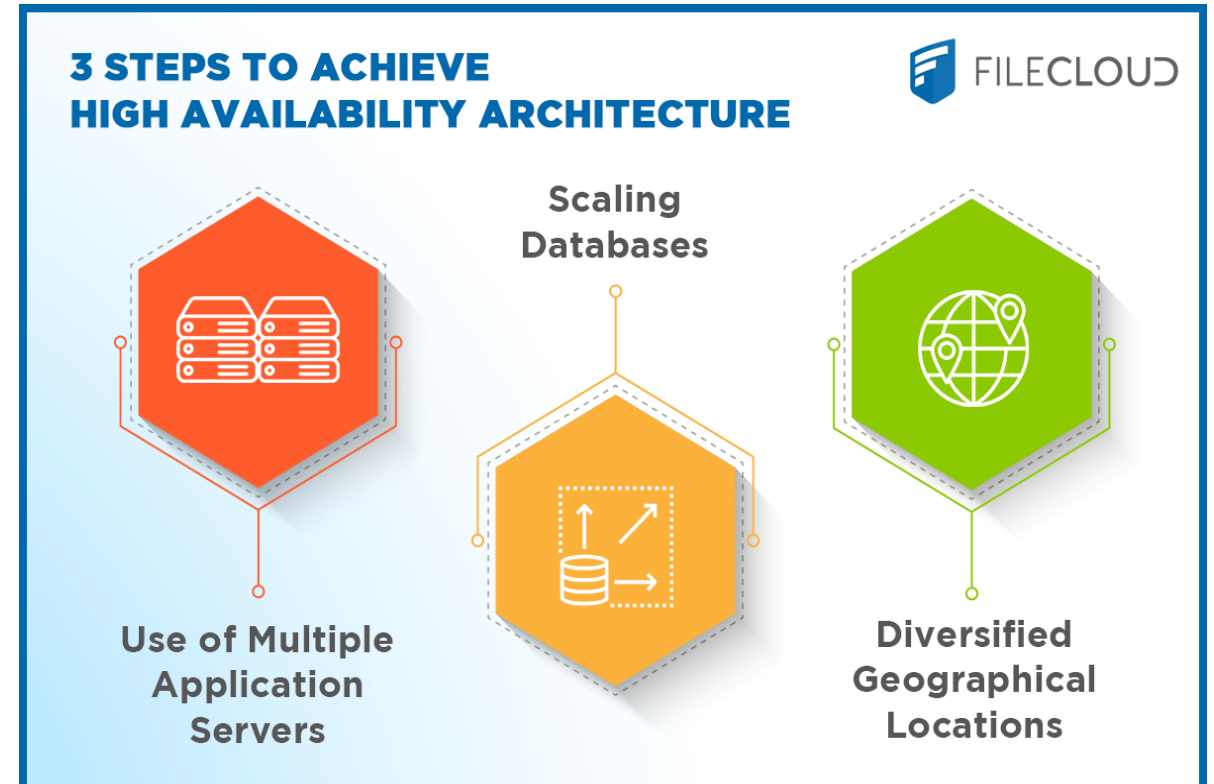
New Requirements – 24/7 Availability

Modern applications:

- Do not tolerate downtime
- Require uninterrupted operation
- Must withstand hardware, network, or software failures
- High availability is no longer optional — it is a fundamental design requirement.

New Requirements – Global Distribution

- Users are geographically distributed:
 - Across continents
 - With different network conditions and latencies
 - Requiring access to the same datasets
- **This necessitates geographically distributed databases with replication and synchronization between data centers.**



New Requirements – Low Latency

- User experience depends on:
 - Millisecond-level responses
 - Immediate data retrieval and updates
 - Predictable performance regardless of load
 - Even small delays result in user or revenue loss, making latency a critical design factor.

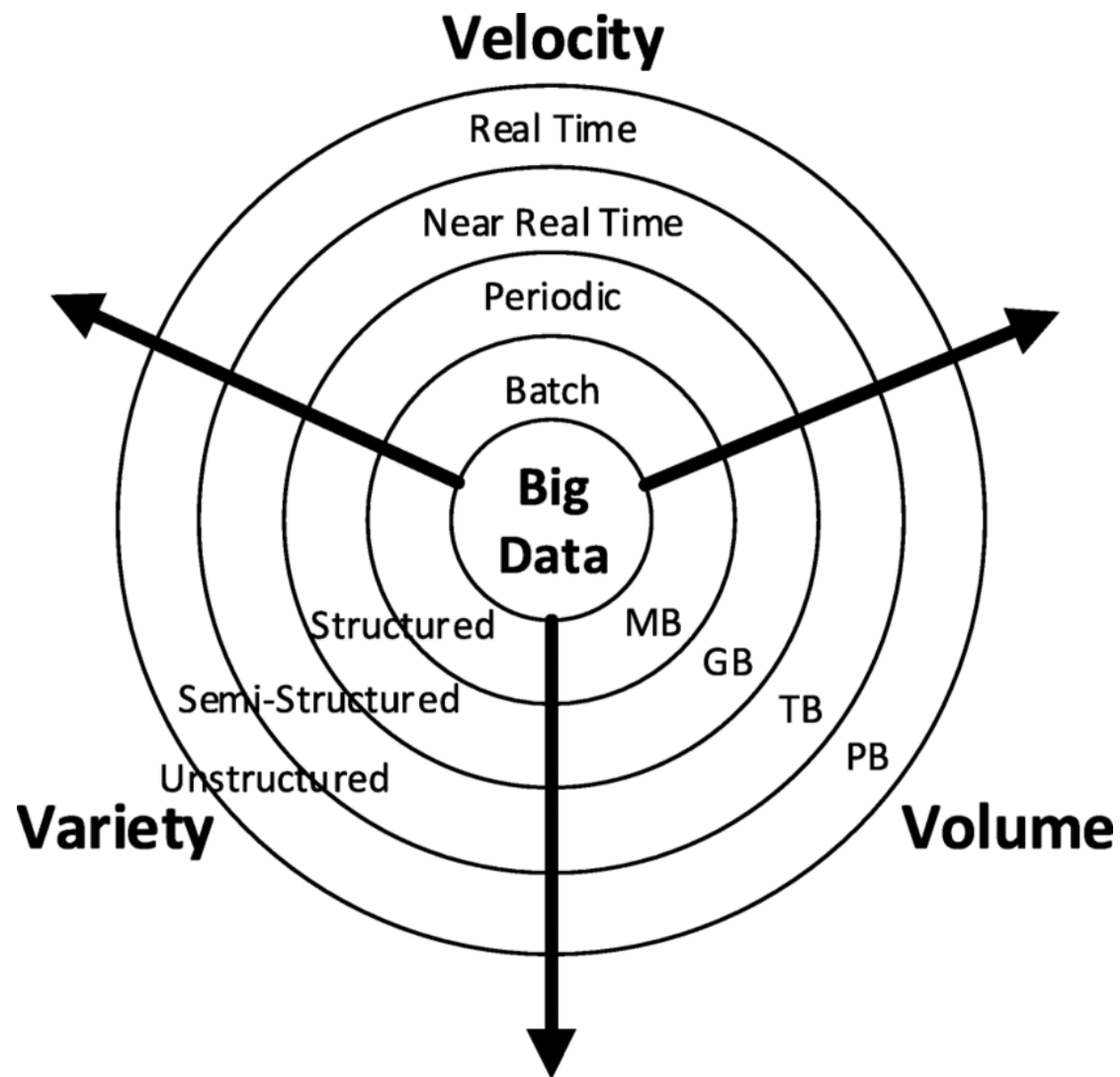
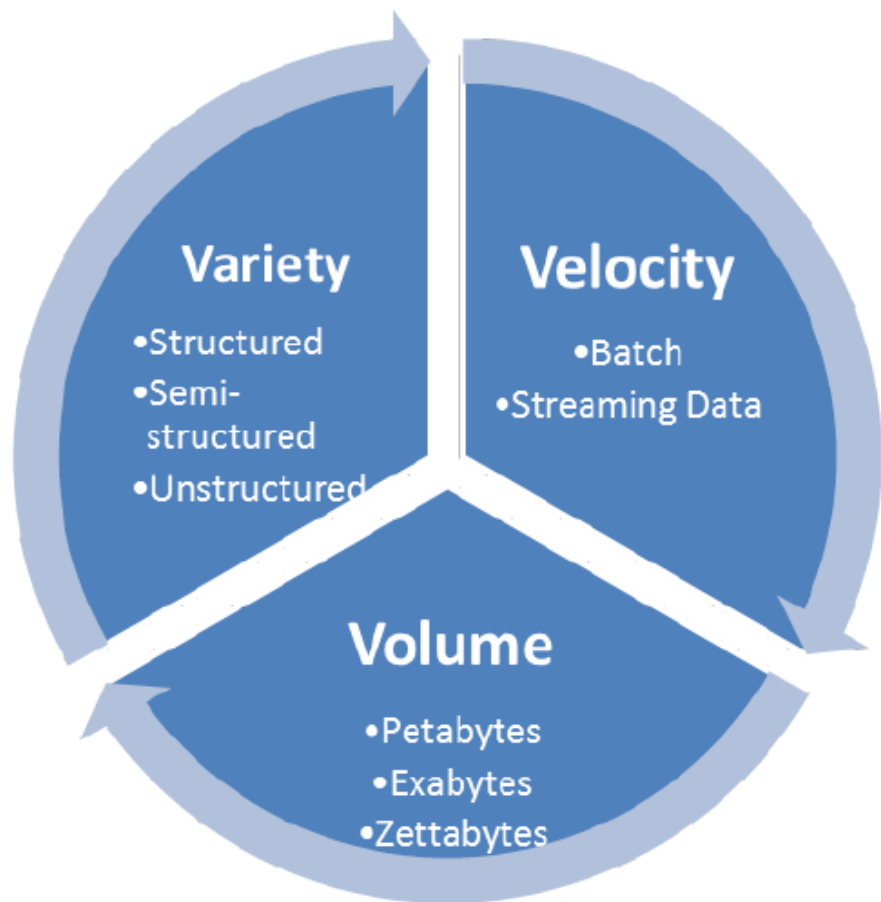


New Requirements – Conclusion

- The new requirements shift the emphasis from **maximum consistency to availability, scalability, and global performance.**
- This gap between requirements and traditional relational databases explains why alternative data architectures emerged — designed from the outset for **large-scale distributed systems.**

Big Data — Definition

- The term **Big Data** describes datasets whose **scale, speed, and heterogeneity** exceed the capabilities of traditional data systems in terms of storage, processing, and analysis.
- It is not simply about more data, but about a **qualitative transformation** in the way data are generated, flow, and are utilized.



Big Data – Volume

- Volume refers to:
 - The enormous amount of data generated and stored
 - Data sizes ranging from terabytes to petabytes and beyond
 - Continuously and cumulatively growing datasets
 - This magnitude makes management through traditional monolithic databases impractical.

Big Data – Velocity

- Velocity concerns:
 - The rate at which data are generated and arrive
 - Continuous streaming in real or near-real time
 - The need for immediate processing and response
 - Examples include sensors, user clicks, system logs, and social media activity.

Big Data – Variety

- Variety describes:
 - The coexistence of structured, semi-structured, and unstructured data
 - Different schemas, formats, and models
 - Text, images, video, graphs, time-series data
 - This heterogeneity challenges the rigid structure of traditional relational models.

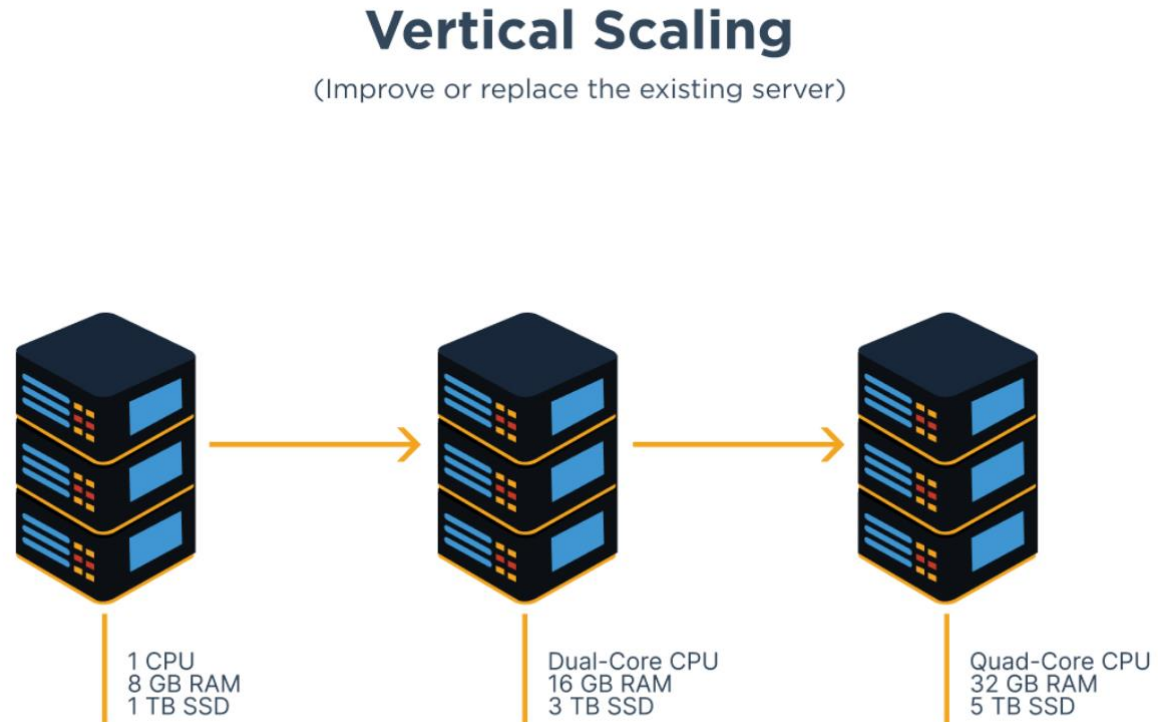
FEATURES	STRUCTURED	SEMI STRUCTURED	UNSTRUCTURED
Format Type	Relational Database	HTML, XML, JSON	Binary, Character
Version Management	Rows, columns, tuples	Not as common – graph is possible	Whole data
Implementation	SQL	Anonymous nodes	-
Robustness	Robust	Limited robustness	-
Storage Requirement	Less	Significant	Large
Applications	DBMS, RDF, ERP system, Data Warehouse, Apache Parquet, Financial Data, Relational Table	Server Logs, Sensor Output	No SQL, Video, Audio, Social Media, Online Forums, MRI, Ultrasound

Vertical vs Horizontal Scaling

- Scalability is a critical issue in modern data systems.
- There are two fundamentally different approaches:
 - Vertical scaling (scale-up)
 - Horizontal scaling (scale-out)
- The choice between them has significant technical and economic implications.

Scale-up (Vertical Scaling)

- Vertical scaling is based on upgrading a single server:
 - More CPU
 - More memory
 - Faster disks
 - Characteristics:
 - Simple architecture
 - Single central management point
 - Limited upper scalability bound
 - This approach is historically associated with traditional RDBMS.

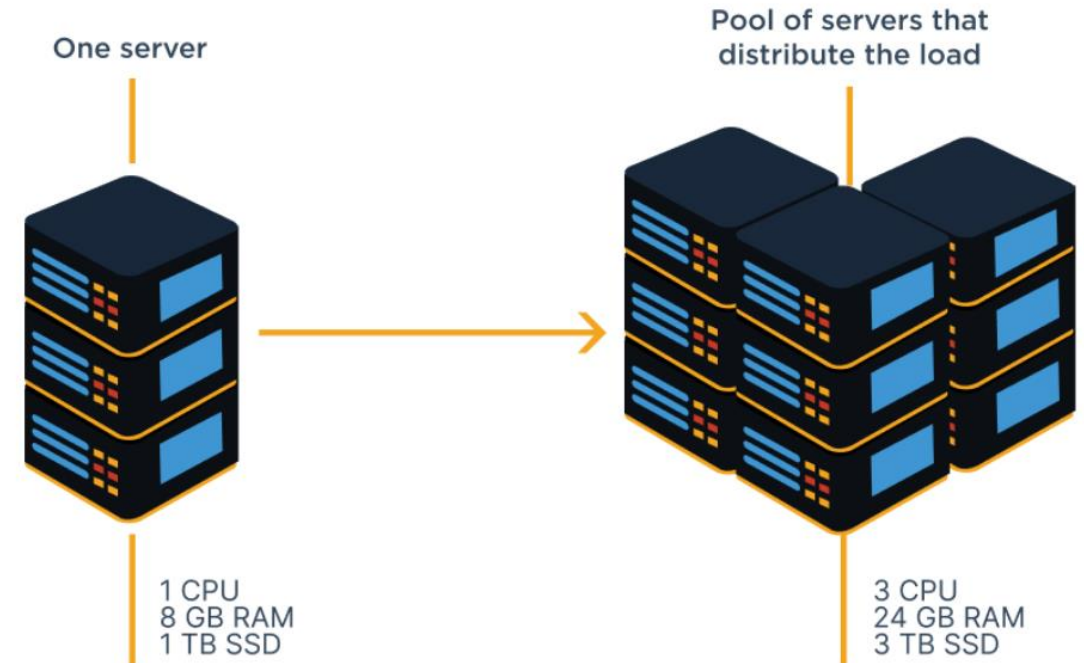


Scale-out (Horizontal Scaling)

- Horizontal scaling is based on adding more servers:
 - Data partitioning (sharding)
 - Replication
 - Parallel processing
 - Characteristics:
 - Theoretically unlimited scalability
 - Fault tolerance against node failures
 - Increased synchronization complexity
 - It constitutes the core design principle of Big Data and NoSQL systems.

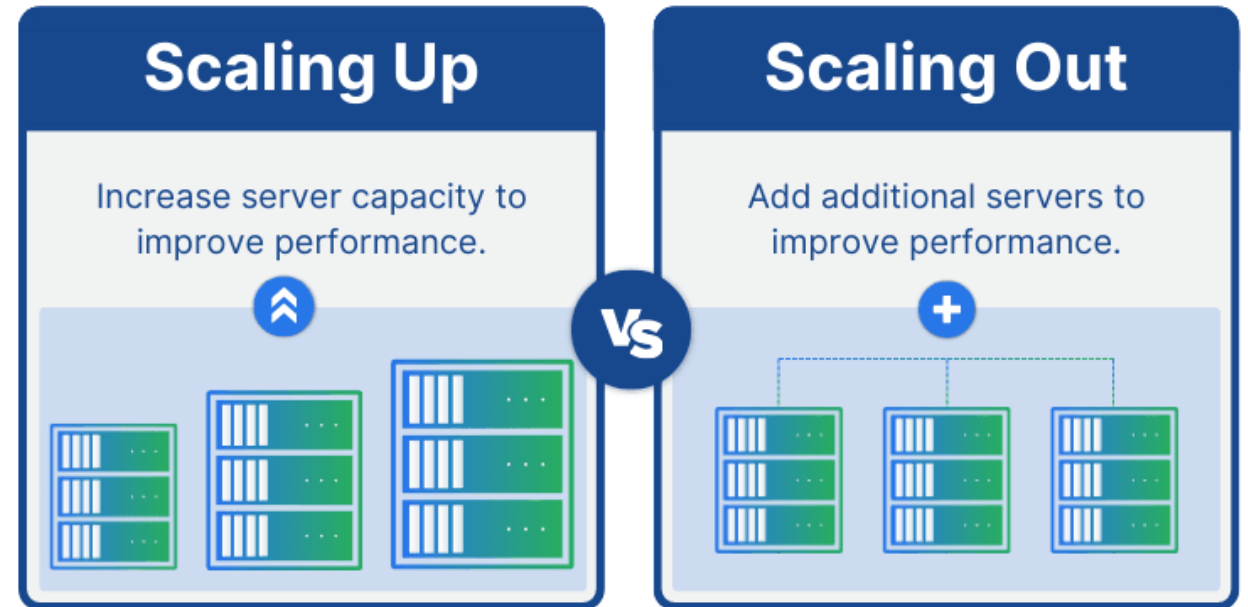
Horizontal Scaling

(Add more same-size nodes)



Vertical vs Horizontal Scaling – Economic Implications

- **Scale-up:**
 - Expensive specialized hardware
 - Exponential cost growth for marginal performance gains
 - Vendor lock-in
- **Scale-out:**
 - Use of commodity hardware
 - Gradual investment based on workload
 - Better cost-performance ratio at large scale



Vertical vs Horizontal Scaling – Technical Implications

- **Scale-up:**
 - Simpler consistency management
 - Limited availability
 - Single point of failure
- **Scale-out:**
 - Distributed data and computation
 - Need for replication and consistency models
 - Introduction of latency and synchronization challenges

Vertical vs Horizontal Scaling – Conclusion

- Traditional RDBMS were primarily designed for **scale-up** environments.
- The world of **Big Data requires scale-out architectures**, where scalability, availability, and fault tolerance are achieved through distributed systems — at the cost of increased complexity.

Limitations of RDBMS

- Although relational database systems remain highly reliable, their design is based on assumptions that do not align with the requirements of modern, distributed, large-scale applications.
- Their core limitations become evident in Big Data environments.

Limitations of RDBMS – Difficulty in Scale-out

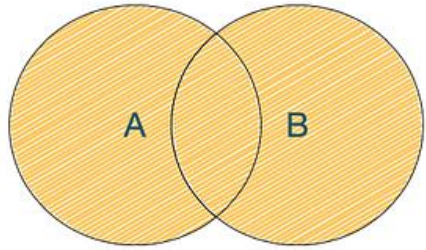
- RDBMS were designed primarily for vertical scaling:
 - Horizontal data distribution is complex
 - Sharding requires manual design
 - Maintaining ACID properties across multiple nodes is expensive
 - As a result, horizontal scalability is not an inherent characteristic of classical RDBMS.

Limitations of RDBMS – Fixed Schema

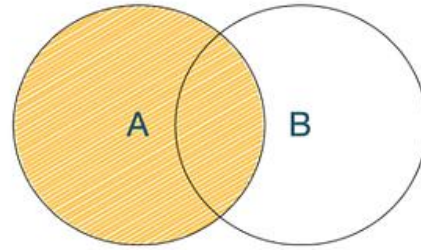
- The strict and predefined schema:
 - Requires full knowledge of data structure in advance
 - Slows rapid application evolution
 - Makes changes costly in large datasets
 - In dynamic and heterogeneous environments, fixed schemas act as a restrictive factor.

Limitations of RDBMS – Expensive Joins

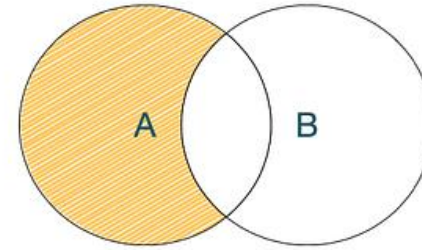
- Joins are a fundamental feature of RDBMS, but:
 - In distributed environments, they require data movement between nodes
 - They significantly increase latency
 - They limit scalability at large scale
 - In Big Data scenarios, joins become a major performance bottleneck.



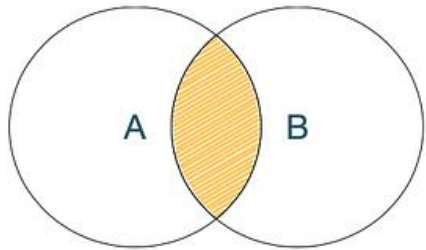
Full Outer Join
Cross Join



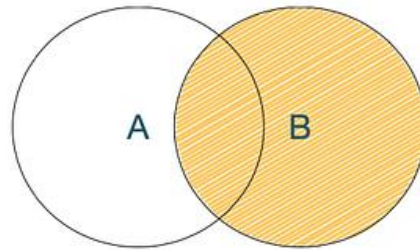
Left Outer Join



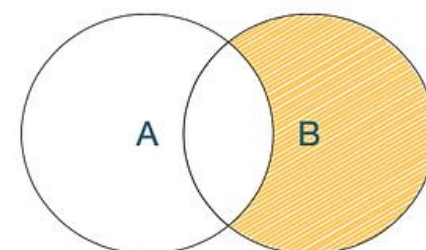
Left Anti Join
Null Aware Left Anti Join



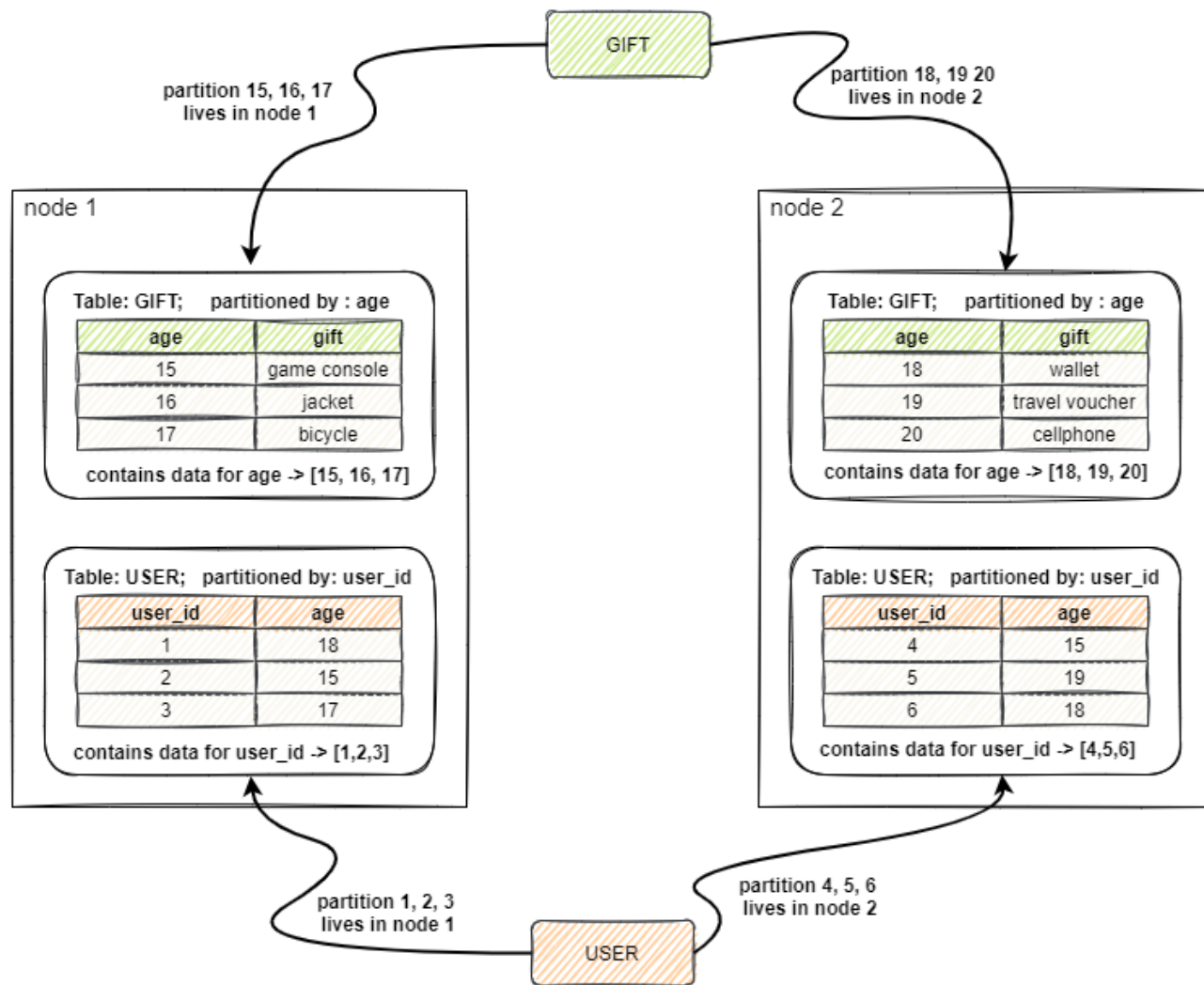
Inner Join
Semi Join



Right Outer Join



Right Anti Join



The join output that we want

user_id	age	gift
1	18	wallet
2	15	game console
3	17	bicycle
4	15	game console
5	19	travel voucher
6	18	wallet

* Type of join is not relevant here as full, left, right, inner join -- all of them yields the same output

Limitations of RDBMS – Complex Distributed Consistency

- Maintaining strong consistency:
 - Requires synchronization across nodes
 - Increases response time
 - Reduces availability during network failures
 - Distributed implementation of ACID guarantees becomes extremely complex and often impractical at global scale.

Limitations of RDBMS – Conclusion

- The limitations of RDBMS are not technological failures, but consequences of their original design goals.
- The gap between those goals and modern requirements explains why alternative data models emerged — designed from the outset for distributed operation, scalability, and flexibility.

The Problem Is NOT SQL

- In discussions around Big Data and NoSQL systems, it is often mistakenly assumed that SQL is the limiting factor.
- In reality, SQL remains one of the most powerful and expressive data management languages.
- The issue does not lie in the language, **but in the underlying data model and architectural implementation** of traditional RDBMS.

SQL Is Powerful

- SQL:
 - Is declarative and expressive
 - Supports complex queries and analytics
 - Provides strict semantics and clear modeling
 - Has decades of maturity and a vast ecosystem
 - This language did not “fail” — it continues to serve as the foundation of many modern systems.

**Σας Ευχαριστώ
(Thank you)**